Technical Approach - Cynbit Discovery Copilot

Ananya Garg | AI/ML Engineer Intern Assessment | 30 October 2026

1. Overview

Cynbit Discovery Copilot is a proof of concept that converts an unstructured enquiry into a structured discovery brief for human review.

I kept the architecture simple and explainable:

Client Enquiry -> Classification -> Knowledge Retrieval -> Gemini Analysis -> Validation & Grounding -> Human Review

The system uses:

- rule-based classification for enquiry routing;
- TF-IDF and cosine similarity for information retrieval;
- Gemini for generating the discovery brief;
- Pydantic for validating structured output;
- additional grounding checks for unsupported recommendations and claims; and
- Streamlit for the user interface.

I did not train or fine-tune a model because the task did not require a custom predictive model. I also did not use a vector database because the current knowledge base is small.

2. Main Components

Component	Purpose	Main File
Classifier	Routes enquiries into Business Sales Lead, Non-Sales Enquiry, or Not Applicable.	src/classifier.py
Retriever	Finds relevant Cynbit services, products and case studies using TF-IDF.	src/retriever.py
Prompting	Builds the evidence-grounded instructions for Gemini.	src/prompts.py
Analyzer	Runs the end-to-end analysis and Gemini call.	src/analyzer.py
Schema	Validates structured model output using Pydantic.	src/schemas.py
Grounding	Checks generated recommendations and potentially unsupported wording.	src/grounding.py
Web App	Provides the interactive Streamlit interface.	app/streamlit_app.py

3. Data Pipeline

I created a small knowledge base from Cynbit's public website. It contains information about services, products, case studies, Cynbit's process, BitElevate and basic company information.

Field	Purpose

id	Unique identifier for the knowledge entry.
type	Category such as service, product, case study, process or program.
title	Name of the service, product or case study.
text	Website information stored in a short searchable form.
source_url	Public Cynbit page used as the source.
source_name	Readable name of the source page.
retrieval_hints	Extra search terms added to improve matching; not treated as Cynbit claims.

For business enquiries, the retrieval pipeline mainly searches services, products and case studies. The knowledge base is maintained manually for this proof of concept.

4. Enquiry Classification

I used deterministic rules for the first routing step instead of asking the LLM to classify every enquiry.

Output	Meaning
Business Sales Lead	A business or technology requirement that can continue to retrieval and LLM analysis.
Non-Sales Enquiry	Career, internship, training or similar enquiry that should not enter the sales workflow.
Not Applicable	An unrelated request that does not match the intended business scope.

This makes basic routing predictable, inexpensive and easy to test. It also avoids unnecessary LLM calls for obvious career or unrelated enquiries.

5. Knowledge Retrieval

For retrieval I used scikit-learn's TfidfVectorizer. Each knowledge entry is searched using its title, text and retrieval hints. The vectorizer uses lowercase text, English stop-word removal, unigrams and bigrams.

The client enquiry is transformed using the same vectorizer. Cosine similarity is calculated between the enquiry and each knowledge entry, and small weights are applied by document type. The highest-scoring documents are passed to the LLM.

Why TF-IDF?

I considered embedding-based retrieval, but the current knowledge base is small. TF-IDF is simple, fast, free and easy to debug. Its main limitation is that it depends more on word overlap than semantic meaning. If the knowledge base grows or enquiries use more varied language, embedding-based retrieval would be worth comparing.

6. LLM and Prompt Engineering

Gemini is used only after classification and retrieval. Instead of giving the model the entire knowledge base, the application provides the client enquiry and only the retrieved Cynbit entries.

The model generates a structured discovery analysis containing:

- enquiry type and requirement summary;
- lead priority;
- recommended services and relevant products;
- relevant case studies;
- discovery questions;
- recommended next action; and
- a draft response.

The prompt instructs the model to use only the supplied evidence and not invent services, products, case studies, prices, timelines, guarantees or unsupported capabilities. Case studies are treated as related published examples rather than proof that Cynbit solved exactly the same client problem. Lead priority is an AI-assisted heuristic, not an official Cynbit sales rule.

7. Validation and Grounding

The model output is returned as structured JSON and validated using Pydantic. After generation, the application performs additional grounding checks. Recommendations can be checked against the retrieved documents, and the application can flag potentially unsupported wording such as guarantees, experience claims, prices or timelines.

These checks reduce obvious errors but do not prove that every generated sentence is correct. The final output is therefore always presented for human review rather than being sent automatically.

8. Evaluation

I created manually written test enquiries covering document automation, legacy migration, SEO, review automation, cloud migration, CRM automation, internship enquiries, unrelated requests and autonomous AI-agent requirements.

Component	Result on Original 10-Case Set
Classification	10/10
Service retrieval	10/10
Product retrieval	10/10
Case-study retrieval	10/10

A second set of 18 enquiries was also used to test additional classifier behaviour and retrieval cases. These are manually created development tests, not an independent benchmark, so I do not treat these results as general model accuracy.

I also generated sample Gemini outputs and reviewed them against the retrieved evidence to identify unsupported wording and improve the grounding rules.

9. User Interface

The proof of concept is implemented as a Streamlit application. The user enters an enquiry and the application shows the classification, retrieved evidence, retrieval scores, discovery analysis, recommended services/products, related case studies, discovery questions, next action and a draft response for review.

Career enquiries and unrelated enquiries can be handled without calling the LLM. If the Gemini API is unavailable, the deterministic classification and retrieval stages can still be used.

10. Execution Plan

Stage	Work
1. Validation	Collect a small set of past enquiries with permission, remove sensitive information, and ask Cynbit staff to label the correct type, relevant service and priority.
2. Evaluation	Measure classification quality, retrieval quality, service recommendation quality and how much staff edit generated drafts.
3. Improvement	Improve the knowledge base, tune retrieval, compare TF-IDF with semantic embeddings if needed, and refine prompts/grounding checks.
4. Pilot	Run the system alongside the existing enquiry process while employees continue to review and send responses.
5. Integration	Only after validation, consider integration with the website enquiry form or CRM.

11. Limitations

- The knowledge base was created manually from public website information.
- TF-IDF may miss semantically similar enquiries that use very different wording.
- The classifier uses rules and keywords rather than a trained intent model.
- Evaluation cases were written manually during development.
- No real Cynbit client enquiries were used.
- Lead priority has not been validated against actual sales outcomes.
- LLM output still requires human review.

12. How to Run

Install dependencies:

```
pip install -r requirements.txt
```

Create a .env file from .env.example and add a Gemini API key.

Run the Streamlit application:

```
streamlit run app/streamlit_app.py
```

The deterministic evaluation can be run separately using the test files included in the repository.